

Ejercicios de clase

Problema 1: Propagación Hacia Adelante (Cálculo Manual Detallado)

Considere una red neuronal simple de dos capas (una capa de entrada, una capa oculta y una capa de salida) con la siguiente arquitectura:

- **Capa de Entrada (Capa 0):** 2 neuronas (x_1, x_2).
- **Capa Oculta (Capa 1):** 2 neuronas (h_1, h_2).
- **Capa de Salida (Capa 2):** 1 neurona (y_1).

La función de activación para las capas oculta y de salida es la **función sigmoide logística**:

$$f(z) = \frac{1}{1+e^{-z}}.$$

Se proporcionan los siguientes pesos y sesgos (biases):

- **Pesos de Capa 0 a Capa 1 ($W^{(1)}$):** $W^{(1)} = \begin{pmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \end{pmatrix} = \begin{pmatrix} 0.5 & -1.0 \\ 0.8 & 0.2 \end{pmatrix}$ (Donde w_{ij} es el peso desde la entrada x_i a la neurona oculta h_j)
- **Sesgos de Capa 1 ($b^{(1)}$):** $b^{(1)} = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = \begin{pmatrix} 0.1 \\ -0.3 \end{pmatrix}$
- **Pesos de Capa 1 a Capa 2 ($W^{(2)}$):** $W^{(2)} = \begin{pmatrix} w_{11} \\ w_{21} \end{pmatrix} = \begin{pmatrix} 1.2 \\ -0.7 \end{pmatrix}$ (Donde w_{ij} es el peso desde la neurona oculta h_i a la neurona de salida y_j)
- **Sesgo de Capa 2 ($b^{(2)}$):** $b^{(2)} = (b_1) = (0.4)$

Tarea: Dado el vector de entrada $X = \begin{pmatrix} 0.7 \\ 0.3 \end{pmatrix}$, calcule el valor exacto de la salida de la red y_1 .

Realice todos los cálculos intermedios (entradas netas z y activaciones a) para cada neurona.

Problema 2: Demostración del Descenso del Gradiente

El objetivo del descenso del gradiente es encontrar un vector de pesos W que minimice una función de error $E(W)$. La regla de actualización se da como $W_{nuevo} = W_{viejo} + \Delta W$.

Queremos encontrar el ΔW que *más* reduzca el error.

Tarea: Usando una **expansión de Taylor de primer orden** para la función de error $E(W + \Delta W)$ alrededor de W , demuestre matemáticamente que la dirección de ΔW que produce la máxima disminución en E (para un paso de tamaño pequeño η) es la dirección opuesta al gradiente, $\nabla E(W)$. Es decir, demuestre que $\Delta W = -\eta \nabla E(W)$ es la elección óptima.

Problema 3: Derivación de la Regla de la Cadena (Capa de Salida)

Considere una red donde la función de error total es el **Error Cuadrático Medio (SSE)** para un solo ejemplo de entrenamiento: $E = \frac{1}{2}(y_{deseado} - y_{actual})^2$

Donde $y_{actual} = f(z_j)$ es la activación de una neurona de salida j , y $z_j = \sum_i w_{ij}h_i + b_j$ es su entrada neta (donde h_i es la activación de una neurona oculta i de la capa anterior).

Tarea: Usando la regla de la cadena, derive la expresión para el gradiente del error con respecto a un peso de la capa de salida, w_{ij} . Es decir, encuentre $\frac{\partial E}{\partial w_{ij}}$. Descomponga la derivada en tres partes claras, como se discutió en la clase ($\frac{\partial E}{\partial y_{actual}}$, $\frac{\partial y_{actual}}{\partial z_j}$, y $\frac{\partial z_j}{\partial w_{ij}}$).

Problema 4: Derivación de la Regla Delta (Capa Oculta)

Este es el núcleo de la retropropagación. Ahora queremos encontrar el gradiente para un peso w_{ki} que conecta una neurona de entrada x_k a una neurona oculta h_i . El error E depende de w_{ki} *indirectamente* a través de todas las neuronas de la capa de salida a las que h_i está conectada.

Tarea: Derive la expresión para $\frac{\partial E}{\partial w_{ki}}$. Demuestre cómo el gradiente depende de la suma ponderada de los "deltas" (δ_j) de la capa de salida. Asuma una red con múltiples neuronas de salida j .

Sugerencia: Comience con $\frac{\partial E}{\partial w_{ki}} = \frac{\partial E}{\partial h_i} \frac{\partial h_i}{\partial z_i} \frac{\partial z_i}{\partial w_{ki}}$. El término clave a encontrar es $\frac{\partial E}{\partial h_i}$.

Problema 5: Derivada de la Función de Activación (Tanh)

En la clase se demostró que si $f(z) = \text{sigmoide}(z)$, entonces $f'(z) = f(z)(1 - f(z))$. Esta elegante propiedad simplifica los cálculos. Otra función de activación popular es la **tangente hiperbólica (tanh)**: $f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

Tarea:

1. Calcule la derivada de la función $\tanh(z)$, es decir, $f'(z)$.
2. Demuestre que esta derivada puede expresarse de forma compacta en términos de su propia salida, $f(z)$. (Respuesta esperada: $f'(z) = 1 - [f(z)]^2$).

Problema 6: Retropropagación (Cálculo Manual Completo)

Considere la red más simple posible: 1 entrada (x_1), 1 neurona en una capa oculta (h_1), y 1 neurona de salida (y_1).

- Función de activación (oculta y salida): Sigmoide, $f(z) = \frac{1}{1+e^{-z}}$.
- Función de error: $E = \frac{1}{2}(y_{deseado} - y_{actual})^2$.

Valores Iniciales:

- Entrada: $x_1 = 1.0$
- Salida Deseada: $y_{deseado} = 0.5$
- Pesos iniciales: $w_1 = 0.4$ (de x_1 a h_1), $w_2 = 0.8$ (de h_1 a y_1).
- (Para simplificar, ignore los sesgos, $b_1 = 0$, $b_2 = 0$).

- Tasa de aprendizaje (learning rate): $\eta = 0.1$

Tarea: Realice una iteración completa del algoritmo de retropropagación.

1. **Forward Pass:** Calcule h_1 (salida de la neurona oculta) y y_1 (salida final).
2. **Cálculo de Error:** Calcule el error E .
3. **Backward Pass (Deltas):**
 - Calcule el "delta" de la capa de salida: $\delta_{y_1} = (y_{actual} - y_{deseado}) \cdot f'(z_{y_1})$.
 - Calcule el "delta" de la capa oculta: $\delta_{h_1} = \delta_{y_1} \cdot w_2 \cdot f'(z_{h_1})$.
4. **Actualización de Pesos:**
 - Calcule Δw_2 y el nuevo w_2 .
 - Calcule Δw_1 y el nuevo w_1 .

Problema 7: Análisis de la Regla Delta Generalizada (Momentum)

La regla de actualización con momentum es:

$$\Delta w(n) = -\eta \frac{\partial E}{\partial w}(n) + \alpha \Delta w(n-1)$$

Donde n es la iteración actual y α es el factor de momentum.

Tarea:

1. Expanda algebraicamente la fórmula para $\Delta w(n)$ de forma recursiva hasta $\Delta w(n-3)$. Es decir, sustituya $\Delta w(n-1)$, luego $\Delta w(n-2)$, etc.
2. Observando la expansión, explique conceptualmente cómo este término: a) Ayuda a "suavizar" las oscilaciones cuando el gradiente cambia de signo rápidamente. b) Ayuda a "acelerar" el descenso en regiones donde el gradiente es pequeño pero constante (mesetas).

Problema 8: Comparación de Funciones de Error (SSE vs. Cross-Entropy)

El descenso del gradiente puede ser lento si la neurona de salida "satura" (su salida es cercana a 0 o 1). Cuando esto sucede, la derivada de la sigmoide $f'(z)$ se vuelve casi cero, deteniendo el aprendizaje.

Funciones de Error:

1. **SSE:** $E_{SSE} = \frac{1}{2}(y_d - y_a)^2$
2. **Cross-Entropy (CE):** $E_{CE} = -[y_d \ln(y_a) + (1 - y_d) \ln(1 - y_a)]$

Donde $y_a = f(z) = \text{sigmoide}(z)$. El "delta" de la capa de salida es $\delta_j = \frac{\partial E}{\partial y_a} \frac{\partial y_a}{\partial z}$.

Tarea:

1. Calcule $\frac{\partial E_{SSE}}{\partial z}$ para la función de error SSE.
2. Calcule $\frac{\partial E_{CE}}{\partial z}$ para la función de error Cross-Entropy.

3. Compare los dos resultados. Demuestre que el término $f'(z)$ (el causante de la saturación) se cancela en la derivada de Cross-Entropy, y explique por qué esto conduce a un aprendizaje más rápido.

Problema 9: Topología y Poder de Representación (Conceptual)

En clase se mencionó que una capa oculta puede crear una región convexa (delimitada por líneas), pero se necesitan dos capas ocultas para resolver "cualquier problema" creando formas arbitrarias.

Tarea: Usando la analogía de la clase de que "cada neurona dibuja una línea", explique geométricamente:

1. Por qué una sola capa oculta seguida de una neurona de salida (que actúa como una función AND lógica de las líneas) solo puede separar regiones **convexas** del espacio de entrada (por ejemplo, un polígono).
2. Cómo una *segunda* capa oculta permite a la red combinar estas regiones convexas (con funciones OR, XOR, etc.) para formar regiones de decisión **no convexas** e incluso **disjuntas** (por ejemplo, dos círculos separados).

Problema 10: Formulación Matemática de un Autoencoder

La clase introduce los Autoencoders como una red que aprende a reconstruir su propia entrada X . Considere un autoencoder simple con una capa oculta ("bottleneck") y **pesos atados (tied weights)**, donde $W_{\text{decodificar}} = W_{\text{codificar}}^T$.

- Capa de Entrada: $X \in \mathbb{R}^n$
- Capa Oculta: $H \in \mathbb{R}^m$ (donde $m < n$)
- Capa de Salida: $\hat{X} \in \mathbb{R}^n$
- Pesos de codificación: W (matriz $n \times m$)
- Sesgo de codificación: b_1 (vector $m \times 1$)
- Pesos de decodificación: W^T (matriz $m \times n$)
- Sesgo de decodificación: b_2 (vector $n \times 1$)
- Activación (oculta y salida): Sigmoide, $f(z)$.

Tarea:

1. Escriba la ecuación completa para la propagación hacia adelante: primero para la capa oculta H (el "código") y luego para la salida reconstruida $\hat{X}$, en términos de $X, W, b_1, b_2, f()$.
2. Escriba la función de error $E(X, \hat{X})$ usando la Suma de Errores Cuadráticos (SSE).
3. Derive la expresión para el gradiente del error con respecto a la matriz de pesos de codificación, $\frac{\partial E}{\partial W}$. (Nota: Este es un problema muy avanzado que combina *backpropagation* con cálculo matricial).